BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space : The collection of all possible physical states of the environment (for instance, different (x, y) coordinate points on the grid). It is finite and fixed in size.

Search Tree : A tree of hypothetical trails that are being searched out by the search algorithm from the start state. Several different nodes of the search tree can represent the same state of the state space.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

Problem Solved : With the help of a Reached Set, the Tree Search becomes a Graph Search, which avoids repetition of state expansion and also avoids cycling and looping infinitely.

Effect on DFS : If there is no reached set in the DFS algorithm, the DFS agent will loop infinitely between two adjacent nodes
(for example, between (0,0) and (0,1))

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

Optimality of BFS : BFS traverses nodes in a level-by-level manner (from shallowest nodes to deepest nodes). Due to the fact that each step in the grid has a uniform cost of 1, when BFS visits the target for the first time, the path is guaranteed to be the shortest and optimal one.

Suboptimality of DFS : DFS goes deeper in the same path using a LIFO stack until it encounters a wall or reaches a dead end. In doing so, it makes decisions based on arbitrary directions, leading to winding paths.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS/UCS:
> Space Complexity : O(b^d) (Exponential)
> Memory Limitation : Needs to keep all the nodes generated so far in the frontier queue, at the current level of depth exploration. In case of a gigantic 1000 x 1000 grid, the memory usage becomes exponential and quickly runs out of memory.

DFS:
> Space Complexity : O(b * m) (Linear)
> Memory Advantage : Keeps only the current search path and unexplored sibling nodes in the call stack, thereby using up much lesser memory even in case of a large grid.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation